

GATE 2027 — Algorithms: Complete Notes

Weightage: ~8-9 marks

1. Asymptotic Analysis

- Big-O (upper bound), Big-Ω (lower bound), Big-Θ (tight bound)
 - Recurrence relations solved via three methods:
 1. **Substitution method** — guess the solution, prove by induction
 2. **Recursion tree method** — expand each level's work, sum across all levels
 3. **Master Theorem** — for $T(n) = aT(n/b) + f(n)$:
 - Compare $f(n)$ with $n^{\log_b a}$
 - **Case 1:** $f(n) = O(n^{\log_b a - \epsilon}) \rightarrow T(n) = \Theta(n^{\log_b a})$
 - **Case 2:** $f(n) = \Theta(n^{\log_b a}) \rightarrow T(n) = \Theta(n^{\log_b a} \cdot \log n)$
 - **Case 3:** $f(n) = \Omega(n^{\log_b a + \epsilon})$, and regularity condition holds $\rightarrow T(n) = \Theta(f(n))$
 - **Special forms ($T(n) = aT(\sqrt{n}) + f(n)$):** substitute $n = 2^m$ to convert into a standard recurrence in m , apply Master Theorem, then substitute $m = \log n$ back into the final answer.
-

2. Sorting Algorithms

Algorithm	Best	Average	Worst	Stable	In-place	Notes
Bubble Sort	n	n^2	n^2	Yes	Yes	rarely optimal but simple
Insertion Sort	n	n^2	n^2	Yes	Yes	fast for nearly-sorted data
Selection Sort	n^2	n^2	n^2	No	Yes	always n^2 comparisons
Merge Sort	$n \log n$	$n \log n$	$n \log n$	Yes	No	needs $O(n)$ extra space
Quick Sort	$n \log n$	$n \log n$	n^2	No	Yes	worst case with bad pivot choice
Heap Sort	$n \log n$	$n \log n$	$n \log n$	No	Yes	uses a max/min-heap

- **Quick Sort:** partition logic — pick pivot, place smaller elements left, larger right; worst case occurs with already-sorted input and a poor pivot strategy (e.g., always picking first/last element)
 - **Merge Sort:** divide-and-conquer, always splits in half, merges two sorted halves in $O(n)$
 - **Lower bound for comparison-based sorting:** $\Omega(n \log n)$ — proven via decision tree argument ($n!$ possible orderings, $\log(n!) = \Theta(n \log n)$ comparisons needed)
 - **Non-comparison sorts:** Counting Sort $O(n+k)$, Radix Sort $O(d(n+k))$, Bucket Sort $O(n)$ average — useful when input has bounded range
-

3. Greedy Algorithms

- **Principle:** make the locally optimal choice at each step, hoping it leads to a globally optimal solution — works only when the problem has the greedy-choice property + optimal substructure
 - **Kruskal's Algorithm (MST):** sort edges by weight, add if it doesn't form a cycle (use Union-Find); $O(E \log E)$
 - **Prim's Algorithm (MST):** grow the tree one vertex at a time, always picking the minimum weight edge connecting tree to non-tree vertex; $O(E \log V)$ with a min-heap
 - **Huffman Coding:** build a binary tree bottom-up, always merging the two lowest-frequency nodes — used for optimal prefix-free codes
 - **Activity Selection:** sort by finish time, greedily pick the next activity that starts after the last selected one finishes
 - **Dijkstra's Algorithm (shortest path, non-negative weights):** greedily picks the minimum-distance unvisited vertex each step; $O((V+E) \log V)$ with a min-heap
-

4. Dynamic Programming

- **Principle:** applicable when a problem has both optimal substructure AND overlapping subproblems; solve subproblems once, store results (memoization or tabulation)
 - **0/1 Knapsack:** $dp[i][w] = \max \text{ value using first } i \text{ items with capacity } w$; recurrence: $dp[i][w] = \max(dp[i-1][w], \text{value}[i] + dp[i-1][w - \text{weight}[i]])$ if $\text{weight}[i] \leq w$
 - **Longest Common Subsequence (LCS):** $dp[i][j] = dp[i-1][j-1] + 1$ if chars match, else $\max(dp[i-1][j], dp[i][j-1])$
 - **Matrix Chain Multiplication:** $dp[i][j] = \min \text{ over all split points } k \text{ of } dp[i][k] + dp[k+1][j] + \text{cost of multiplying resulting matrices}$
 - **Edit Distance:** $dp[i][j] = dp[i-1][j-1]$ if chars match; else $1 + \min(\text{insert, delete, replace costs})$
 - **Coin Change / subset-sum style problems:** similar 2D DP table approach, watch for "minimum coins" vs "number of ways" variants (different recurrences)
-

5. Graph Algorithms

- **Dijkstra's Algorithm:** single-source shortest path, non-negative weights only, $O((V+E) \log V)$ with min-heap
 - **Bellman-Ford Algorithm:** handles negative weights (but not negative cycles), $O(V \cdot E)$ — relax all edges $V-1$ times
 - **Floyd-Warshall Algorithm:** all-pairs shortest path, $O(V^3)$, works with negative weights (not negative cycles), uses dynamic programming over intermediate vertices
 - **Topological Sort:** only possible on a DAG (Directed Acyclic Graph); via DFS (finish-time based) or Kahn's algorithm (in-degree based, using a queue)
 - **Strongly Connected Components:** Kosaraju's or Tarjan's algorithm, both $O(V+E)$
 - **Minimum Spanning Tree:** Kruskal's (edge-based, Union-Find) and Prim's (vertex-based, priority queue) — both $O(E \log V)$ with efficient data structures
-

6. Divide and Conquer

- **Binary Search:** $O(\log n)$
- **Merge Sort, Quick Sort:** covered above
- **Strassen's Matrix Multiplication:** $O(n^{2.81})$, better than naive

- $O(n^3)$ — occasionally tested conceptually (not full derivation)
 - **Closest Pair of Points:** $O(n \log n)$ via divide and conquer — usually only conceptual-level questions in GATE
-

7. NP-Completeness (conceptual, low-frequency but occasionally tested)

- P: solvable in polynomial time
 - NP: verifiable in polynomial time
 - NP-Complete: in NP, and every NP problem reduces to it in polynomial time
 - NP-Hard: at least as hard as NP-Complete, but not necessarily in NP itself
 - Common NP-Complete problems: SAT, Vertex Cover, Hamiltonian Cycle, TSP (decision version), Subset Sum
-

8. PYQ Patterns & Solving Speed Tips

- Recurrence-relation time complexity questions are near-guaranteed every year — practice Master Theorem cases and the $n=2^m$ substitution trick until automatic.
 - Greedy vs DP identification: check if the problem has overlapping subproblems (needs a table/memoization) — if not, and a greedy/local choice can be proven optimal, greedy suffices. When uncertain in an MCQ, DP is the safer general-purpose technique.
 - MST numericals (Kruskal's/Prim's): draw the graph and process edges/vertices by hand — most errors come from missing a cycle-check, not from the algorithm concept.
 - Dijkstra vs Bellman-Ford: check for negative edge weights in the question — this alone often determines the correct algorithm choice in a conceptual MCQ.
 - DP table-filling questions: fill the base cases first (row/column 0), then verify the recurrence direction (are you filling row-wise or column-wise) before continuing — a common trap is inconsistent fill order.
-

9. Common Mistakes to Avoid

- Applying Master Theorem directly to recurrences with $\sqrt{n}$ terms without first substituting $n = 2^m$
- Assuming Quick Sort is always $O(n \log n)$ — worst case is $O(n^2)$ with poor pivot selection
- Confusing “minimum spanning tree” (Kruskal/Prim) uses with “shortest path” (Dijkstra) uses — different problems, different algorithms
- Forgetting Bellman-Ford is needed (not Dijkstra) when negative edge weights are present
- In DP, forgetting to initialize base cases correctly, leading to an off-by-one error propagating through the whole table